Basic Data Types in Python

DATA 118 Lab — Student Version

Goals

The goal of this tutorial is to introduce you to the style of tutorials we will be using to learn to program and to review some of what you've learned about basic data types in Python. We will cover:

- Ints
- Floats
- Booleans
- Strings

How Does This Tutorial Work?

Before we ask you to start working in a Jupyter notebook, we will ask you to complete this tutorial in a web browser so you can get used to one thing at a time.

Below is an example of a *cell*. Type some basic Python code into a cell. Remember that Python uses built-in operators to do math calculations if you can't think of anything else to type.

Exercise — write your code here:

Now, click on the light blue "Run Code" button at the top of the cell. Hopefully, you should see some output in a separate box just below the cell.

You can type anything into the cell, but usually, you should be typing code to answer to a question we are asking. For example, write some code to answer the question "What is two plus two" in the cell below:

Exercise — write your code here:

This time, you should see a light "Submit Answer" button at the top of the cell. Click on it–what do you see? If you wrote something like `2+2`, you should see a green box saying that you got the right answer.

Now, write some code that will tell you anything BUT the answer to the question "What is two plus two."

Exercise — write your code here:

Click on the "Submit Answer" button. This time, you should see a red box saying that your answer was incorrect. Most of the time, this means that you should revisit your code–you can submit the answer multiple times until you get the right answer. For now, you can move on!

Sometimes, the cells expect your code to match ours exactly. For example, edit the code in the cell below to give the answer to $2 + 2 - 3 + 3$ and click on the "Submit Answer" button.

Exercise

```
2+2
```

Even though `2+2` and `2+2-3+3` give the same value, `4`, the tutorial is telling you that you didn't get the right answer because your code is not an exact match. In this class, we will usually use exact code matching so that you can learn the basics, but there might be times where there are multiple ways to get the right answer. You are always welcome to experiment and ask us for help!

Sometimes, you might see a yellow box with a strange error. If you do, please know that you are likely not making a mistake and we need to fix the issue on the back end. If you see something like this, please let your instructor know!

Syntax Highlighting

Remember that syntax is "the arrangement of words and phrases to create well-formed sentences in a language", a.k.a., coding grammar. Syntax highlighting displays code in different colors and fonts to help programmers identify different pieces of code quickly.

The syntax highlighting on my slides is different from the syntax highlighting in this tutorial, which may be different from the syntax highlighting you are using. Let's take a quick look at the syntax highlighting in your tutorial.

Exercise

```
2+2
"Hello, my name is Professor Nussbaum."
7.7
True
```

Question 1. What colors do you see? Check all that apply:
☐ Black
☐ Dark Blue
☐ Gray
☐ Green
☐ Purple
☐ Red
☐ Light Blue
☐ Yellow

Our 4 lines of code are not colored differently just for fun–the colors are meant to help us identify something.

Question 2. What do you think the colors are meant to signify? Check all that apply:
☐ Longer inputs are green.
☐ Numbers are dark blue.
☐ All capitalized inputs are light blue.
☐ Strings are green.
☐ Gray is used for errors.
☐ True and False are light blue.

Discuss with a neighbor (or on Ed):
Open an empty Jupyter notebook and check to see what colors `ints`, `floats`, strings and Booleans are on your own computer. Is it the same color as your neighbor?

Ints and Floats

Remember that numbers can be stored as `ints` or `floats` in Python. An `int` is a number without a decimal point; a `float` usually has a decimal point (or is stored in scientific notation). Remember also that the `type()` function will tell us what kind of object we are working with.

Edit the code below to find the types of the numbers:

Exercise

```
18
22.7
```

Question 3. What is the type of `18`?

☐ `Boolean`
☐ `int`
☐ `float`
☐ `str`, the abbreviation for string

Question 4. What is the type of `22.7`?

☐ `Boolean`
☐ `int`
☐ `float`
☐ `str`, the abbreviation for string

Other types have more specific methods, but for ints and floats, we are happy using simple operators or comparison operators.

Operation	Operator	Example	Output
Addition	`+`	`2+3`	`5`
Subtraction	`-`	`2-3`	`-1`
Multiplication	`*`	`2*3`	`6`
Division	`/`	`7/3`	`2.33`
Floor Division	`//`	`7//3`	`2`
Remainder (mod)	`%`	`7%3`	`1`
Exponentiation	`**`	`2**0.5`	`1.41421`

Use the cell below to experiment with the operators. Try to use the operators in sequence, with and without parentheses.

Exercise — write your code here:

Question 5. Does Python follow PEMDAS for the order of operations?
☐ Yes
☐ No

Question 6. What is the output (including the type) of `6/2`?
☐ `3`, which is an `int`
☐ `3.0`, which is a `float`

Don't forget, there are also the comparison operators:

Comparison	Operator	True Example	False Example
Less than	`<`	`2 < 3`	`2 < 2`
Greater than	`>`	`3 > 2`	`3 > 3`
Less than or equal	`<=`	`2 <= 2`	`3 <= 2`
Greater than or equal	`>=`	`3 >= 3`	`2 >= 3`
Exactly equal	`==`	`3 == 3`	`3 == 2`
Not Equal	`!=`	`3 != 2`	`3 != 3`

Use the cell below to experiment with the comparison operators. Using them with `ints` and `floats` should be familiar, but you can also use them with strings and Booleans.

Exercise — write your code here:

Question 7. What does `"Hello" < "hello"` return?
☐ `True`
☐ `False`

We can use multiple operators, as well as the parentheses, to carry out more advanced calculations. For example, some of you may be familiar with the average or mean:

$$\bar{x} = \frac{x_1 + x_2 + ... + x_n}{n}$$

In other words, to find the average of a variable, you add all the numbers up and divide by the number of observations.

Consider the heights in centimeters of the following Star Wars characters:

- Luke Skywalker–172cm
- C-3PO–167cm
- R2-D2–96cm
- Leia Organa–150cm
- Han Solo–180cm, and
- Chewbacca–238cm

Using the `+` and `/` operators, as well as the parentheses, write a line of code to calculate the average.

Exercise — write your code here:

As we have said, there are not many `int`- and `float`-specific methods, but there are some built-in functions in Python that make working with numbers easier:

Function	Description
`abs()`	Returns the absolute value of a number
`float()`	Returns (or converts an object to) a floating point number
`int()`	Returns (or converts an object to) an integer number
`round()`	Rounds a number

Convert the integer 172 to a float using one of these functions in the cell below:

Exercise

```
172
```

Question 8. Take a closer look at your code. What color is used to highlight a built-in function?

☐ Black
☐ Blue
☐ Gray
☐ Green
☐ Purple
☐ Red
☐ Yellow

The table above is a shortened version of a much larger table with built-in functions.

Discuss with a neighbor (or on Ed):
Click on the link to the larger table. Can you spot any other functions that might be useful?

Variable Assignment

Let's revisit the heights of the Star Wars characters. Consider Chewbacca in particular–it turns out that his height is 228cm, not 238. If we were correcting a data analysis, we would have to re-type in all those numbers to find the correct mean. However, if you were handing it off to a colleague or if enough time had passed, you may not remember exactly why you wrote what you wrote.

Sometimes, to make it easier to track our work, we use something called *variable assignment*. Variable assignment helps us store values to work with using a variable name and the *assignment operator*, `=`.

Consider the following code:

Exercise

```
Chewbacca = 238
```

In this case, the variable name is `Chewbacca`. Notice that it appears in black! Now, edit the code above so that `Chewbacca` is equal to 228, not 238.

This time, if you click on the "Run Code" button, it looks like nothing happens. That's not technically true–we corrected the height by re-storing the value, but we can't necessarily see what happened. Another built-in function is the `print()` function. Click on the "Run Code" button below, and then edit Chewbacca's height–does the output change at all?

Exercise

```
Chewbacca = 238
print(Chewbacca)
```

Now, add some code to assign heights to the rest of our characters, Luke Skywalker, C-3PO, R2-D2, Leia Organa, and Han Solo. Remember, the heights are:

- Luke Skywalker–172cm
- C-3PO–167cm
- R2-D2–96cm
- Leia Organa–150cm
- Han Solo–180cm, and
- Chewbacca–228cm

Exercise

```
Chewbacca = 228
Luke = ...
C3P0 = ...
R2D2 = ...
Leia = ...
Han = ...
```

Variable names *can* be anything, but it's generally good to avoid the names of the built-in functions as well as the Python keywords, listed below:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Note that the keywords ALSO appear as blue.

Booleans

Now let's practice working with Booleans using our variables, `Chewbacca`, `Luke`, `C3P0`, `R2D2`, `Leia`, and `Han`. Remember that Booleans are outputted by the comparison operators:

Comparison	Operator	True Example	False Example
Less than	<	2 < 3	2<2
Greater than	>	3 > 2	3>3
Less than or equal	<=	2 <=2	3<=2
Greater than or equal	>=	3 >= 3	2 >= 3
Exactly equal	==	3 == 3	3 == 2
Not Equal	!=	3 != 2	3 != 3

Is Luke taller than Leia? Write a line of code using the right comparison operator and the variable names. This is a good time to note that our tutorial remembers the variables we assigned earlier!

Exercise — write your code here:

Is Chewbacca shorter than Han Solo? Write a line of code using the right comparison operator and the variable names.

Exercise — write your code here:

It can be helpful to combine some of these comparisons using Python's logical operators. Click on the link and read about these new operators. Then, use them to complete the questions below. You may also find it helpful to use parentheses.

Is Leia taller than *either* of the droids, R2-D2 or C-3PO?

Exercise — write your code here:

Is Chewbacca taller than *both* of the droids, R2-D2 or C-3PO?

Exercise — write your code here:

Strings

Saving the most complicated for last! Unlike `ints`, `floats`, and Booleans, strings have LOTS of methods we can use. Remember that unlike a function, a method is a tool associated with specific type of object.

First, let's create a variable for Leia Organa's full name:

Exercise

```
Leia_fullname = "Leia Organa"
```

A few things:

1. First, we are choosing separate variable name for Leia's full name–if we saved it as `Leia`, we would lose the information about her height. Be careful in your own work not to overwrite variables unless you really, really mean it!
2. `Leia_fullname` is the name of a variable, whereas `"Leia Organa"` is a string stored as `Leia_fullname`. Remember that strings begin and end with quotation marks, and show up as green, whereas variable names are black.

Strings are the same whether you use double or single quotes to define them. Write a line of code to see if `Leia_fullname` is the same as `'Leia Organa'` using a comparison operator.

Exercise — write your code here:

Make sure you are careful to use the single equals sign, `=`, when assigning variables, and the double equals sign, `==`, when checking equality!

Now, write a line of code using a comparison operator to see if `Leia_fullname` is equal to `"leia organa"`.

Exercise — write your code here:

Python does not consider two strings to be the same unless they match exactly. This is where some of the methods come in. The `.lower()` method should convert all of the characters in a string to lowercase. Try and use it with our variable name in the cell below. Save the lowercase version as a new variable, `Leia_lower`, and check to see if it is equal to `"leia organa"`.

Exercise

```
leia_lower = Leia_fullname...
...
```

Now, use the `.capitalize()` method to try and undo what you just did.

Discuss with a neighbor (or on Ed):
Will `.capitalize()` undo what you did completely? If not, what function can undo it?

Exercise

```
leia_lower...
...
```

Like I said, there are many many string methods. Check out a table of string methods–can you find one that will do the opposite of `.lower()`?

Question 9. What is the method that will capitalize all of the letters in a string?
☐ `.isnumeric()`
☐ `.replace()`
☐ `.split()`
☐ `.strip()`
☐ `.upper()`

So far, we have only considered fairly short strings, but they can be relatively long, as well! Consider Princess Leia's message to Obi-Wan Kenobi in *A New Hope*.

Exercise

```
message = "General Kenobi. Years ago you served my father in the Clone
 ↪  Wars. Now he begs you to help him in his struggle against the
 ↪  Empire. I regret that I am unable to present my father's request
 ↪  to you in person, but my ship has fallen under attack, and I'm
 ↪  afraid my mission to bring you to Alderaan has failed. I have
 ↪  placed information vital to the survival of the Rebellion into the
 ↪  memory systems of this R2 unit. My father will know how to
 ↪  retrieve it. You must see this droid safely delivered to him on
 ↪  Alderaan. This is our most desperate hour. Help me, Obi-Wan
 ↪  Kenobi. You're my only hope."
```

We might be interested in locating shorter strings within larger strings. For example, we could examine the contents of `message` to find how many times the shorter string `"Kenobi"` appears, but the longer the string, the more difficult this task will be. Instead, let's consider the `.count()` method. This method is different than many we have seen, because we have to pass what is known as an "argument". An argument is a value that you pass into a function or method–in this case, the argument is the shorter string, `"Kenobi"`.

We can change the argument. Edit the code below so that we count the number of

times the string `"father"` occurs in `message`.

Exercise

```
message.count("Kenobi")
```

Discuss with a neighbor (or on Ed):
Go back to the string methods table and find an interesting method. See if you can use it with `message`.

Exercise — write your code here:

In addition to having specific methods, some of the operators we are already familiar with operate differently with strings. Consider the following cells. What happens if you add the two strings together?

Exercise

```
obiwan = "These aren't the droids you're looking for."
stormtrooper = "These aren't the droids we're looking for."
...
```

When used with strings, the `+` is the concatenation operator. It pastes the two messages together, exactly as they are–notice that there is no space in between the sentences in the concatenated message, even though we would normally put one there.

What happens if you use the multiplication operator, `*`, with `message_short`? Multiply it by 3.

Exercise

```
message_short = "Help me, Obi-Wan Kenobi. You're my only hope."
...
```

You should notice that the multiplication operator will repeat the string (in this case, three times). What happens if you multiply by 1.5 instead?

Exercise

```
message_short = "Help me, Obi-Wan Kenobi. You're my only hope."
...
```

You should get an error! Remember that sometimes error messages are frustrating, but they are supposed to help us debug our code. In this case, we got a `TypeError`, because we tried to multiply a string (allowed) by an object of the `float` type (not allowed). The multiplication operator only works with type `int`.